

ACTIVIDAD LABORATORIO NO.2
INSTALACIÓN Y CONFIGURACIÓN DE OPENCV 4.10.0 CON JAVA EN APACHE NETBEANS:
IMPLEMENTACIÓN DE RUTINAS DE VISIÓN ARTIFICIAL

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING ROGERIO ORLANDO BELTRAN CASTRO

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
01 DE JUNIO
2026

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
DESARROLLO DEL LABORATORIO	3
1. Objetivos	4
a. Objetivo General.....	4
b. Objetivos Específicos	4
2. Instalación de OpenCV	4
a. Descarga desde el sitio oficial	4
b. Extracción del archivo	5
c. Verificación de la estructura de archivos	6
3. Configuración del Proyecto en NetBeans	7
a. Creación del proyecto.....	7
b. Agregar la librería JAR	8
c. Configuración de la VM Option.....	9
d. Selección de la plataforma Java.....	10
e. Verificación de la instalación	11
4. Rutinas de Visión Artificial	12
a. Rutina 1 - Procesamiento Básico (Rutina1Basico.java).....	12
i. Explicación del Código – Rutina 1 Básica Procesamiento Imágenes OpenCV	12
b. Rutina 2 - Detección de Bordes (Rutina2Bordes.java).....	16
i. Explicación del Código – Rutina 2: Detección de Bordes con Canny y Sobel.....	16
c. Rutina 3 - Detección de Rostros (Rutina3Rostros.java)	20
i. Explicación del Código – Rutina 3: Detección de Rostros con Haar Cascade	20
d. Rutina 4 - Transformaciones Geométricas (Rutina4Transformaciones.java)	24
i. Explicación del Código – Rutina 4: Transformaciones Geométricas Imágenes ...	24
5. Análisis de Resultados	29
CONCLUSIONES DEL LABORATORIO	30
BIBLIOGRAFÍA.....	31
AGRADECIMIENTO	32

INTRODUCCIÓN

La percepción computacional constituye una de las áreas más relevantes dentro de la Inteligencia Artificial moderna, ya que permite a los sistemas informáticos adquirir, procesar e interpretar información proveniente del entorno a través de sensores, imágenes y secuencias de video. Dentro de este campo, la visión artificial desempeña un papel fundamental al proporcionar mecanismos que permiten a las máquinas analizar imágenes digitales, identificar patrones, reconocer objetos y extraer información útil para la toma de decisiones automatizadas. Estas capacidades han impulsado el desarrollo de aplicaciones en sectores como la industria manufacturera, la medicina, la seguridad, la robótica, los vehículos autónomos y los sistemas inteligentes de monitoreo y control.

Uno de los frameworks más utilizados para el desarrollo de aplicaciones de visión artificial es OpenCV (Open Source Computer Vision Library), una biblioteca de código abierto que proporciona un amplio conjunto de funciones para el procesamiento digital de imágenes, análisis de video, detección de objetos, reconocimiento de patrones y aprendizaje automático. Gracias a su compatibilidad con diversos lenguajes de programación, entre ellos Java, Python y C++, OpenCV se ha convertido en una herramienta ampliamente utilizada tanto en entornos académicos como industriales para la implementación de soluciones basadas en percepción computacional.

El presente laboratorio tiene como propósito realizar la instalación, configuración y validación funcional de OpenCV en un entorno de desarrollo Java, permitiendo al estudiante familiarizarse con las herramientas fundamentales empleadas en aplicaciones de visión artificial. A través del desarrollo de la actividad se documenta detalladamente el procedimiento de descarga de la biblioteca, la configuración de las dependencias necesarias dentro del entorno Apache NetBeans y la integración de las librerías requeridas para la correcta ejecución de aplicaciones basadas en OpenCV.

Como parte práctica del laboratorio, se implementaron diferentes ejemplos de procesamiento de imágenes orientados a comprobar el funcionamiento de la biblioteca. Entre ellos se incluyen la carga de imágenes digitales, la conversión de imágenes a estructuras matriciales, la copia y almacenamiento de archivos gráficos, así como la visualización de imágenes mediante interfaces gráficas desarrolladas en Java. Estas pruebas permiten comprender cómo OpenCV representa internamente la información visual y cómo dicha información puede ser manipulada mediante operaciones programáticas para construir aplicaciones más avanzadas de análisis y reconocimiento visual.

Desde una perspectiva de Ingeniería de Software, el desarrollo realizado permite comprender la integración de bibliotecas externas dentro de aplicaciones Java, la gestión de dependencias, la configuración de entornos de ejecución y la implementación de soluciones modulares orientadas al procesamiento digital de imágenes. Asimismo, proporciona una base sólida para abordar futuros proyectos relacionados con detección de objetos, reconocimiento facial, clasificación de imágenes, inspección industrial automatizada y sistemas inteligentes basados en visión artificial.

El desarrollo del laboratorio se estructura en diferentes etapas que incluyen la descarga de OpenCV desde su sitio oficial, la instalación y descompresión de los archivos requeridos, la configuración de las librerías nativas dentro de Apache NetBeans, la creación del proyecto Java y la implementación de ejemplos funcionales que demuestran las capacidades básicas de la herramienta. Cada una de estas fases es acompañada por evidencias gráficas y explicaciones técnicas que permiten reproducir el procedimiento de manera controlada y verificable.

Finalmente, este trabajo permite consolidar los conceptos fundamentales de percepción computacional y visión artificial estudiados durante la asignatura, evidenciando la importancia de OpenCV como una herramienta esencial para el desarrollo de aplicaciones inteligentes capaces de interpretar información visual. La experiencia adquirida durante la instalación y utilización de la biblioteca constituye una base importante para la comprensión de técnicas más avanzadas de procesamiento de imágenes y análisis visual que serán abordadas en futuras actividades académicas y profesionales.

DESARROLLO DEL LABORATORIO

A continuación, se presenta el desarrollo paso a paso de la instalación, configuración y ejecución de la biblioteca OpenCV, siguiendo un enfoque práctico y estructurado orientado al aprendizaje de los fundamentos de la visión artificial y el procesamiento digital de imágenes. Cada una de las etapas del

laboratorio fue desarrollada de manera progresiva, comenzando con la descarga de los archivos oficiales de OpenCV, la preparación del entorno de desarrollo y la configuración de las librerías necesarias para su correcta integración con Java.

En las siguientes secciones se documentan detalladamente los procedimientos realizados para la instalación de OpenCV en el sistema operativo Windows, la configuración de las dependencias dentro del entorno Apache NetBeans y la implementación de diferentes ejemplos prácticos destinados a verificar el correcto funcionamiento de la biblioteca. Asimismo, se presentan evidencias de la ejecución de los programas desarrollados, incluyendo procesos de carga de imágenes, representación matricial de datos visuales, copia de archivos gráficos y visualización de imágenes mediante interfaces gráficas.

Adicionalmente, se describen los principales conceptos relacionados con la manipulación de imágenes digitales utilizando OpenCV, destacando la forma en que las imágenes son representadas internamente mediante matrices de píxeles y cómo estas estructuras pueden ser utilizadas para realizar futuras tareas de procesamiento, análisis y reconocimiento visual. Finalmente, se exponen los resultados obtenidos durante las pruebas realizadas, evidenciando el correcto funcionamiento del entorno configurado y estableciendo una base sólida para el desarrollo de aplicaciones más avanzadas de percepción computacional y visión artificial.

1. Objetivos

Antes de iniciar el desarrollo del laboratorio, es importante establecer los objetivos que orientan la actividad. Estos permiten definir el alcance del trabajo y las competencias que se pretenden fortalecer mediante la instalación, configuración y utilización de OpenCV. A continuación, se presentan el objetivo general y los objetivos específicos que guiaron la ejecución de la práctica.

a. Objetivo General

El presente laboratorio tiene como propósito instalar y configurar la biblioteca OpenCV 4.10.0 en el entorno de desarrollo Java con Apache NetBeans IDE 28 y Zulu JDK 17, y demostrar cuatro rutinas fundamentales de visión artificial mediante clases Java independientes.

b. Objetivos Específicos

- Descargar e instalar correctamente OpenCV 4.10.0 para Windows 64-bit desde el sitio oficial.
- Configurar la librería opencv-4100.jar y la DLL nativa en un proyecto NetBeans.
- Resolver el error de incompatibilidad de versión Java (UnsupportedClassVersionError) mediante la selección correcta del JDK.
- Implementar cuatro clases Java independientes demostrando rutinas de visión artificial.
- Modificar los parámetros del código base para observar variaciones en los resultados.

2. Instalación de OpenCV

La instalación de OpenCV constituye el primer paso para desarrollar aplicaciones de visión artificial y procesamiento digital de imágenes. En esta etapa se realiza la descarga de la biblioteca desde su sitio oficial, la descompresión de los archivos necesarios y la configuración inicial del entorno de trabajo. Este procedimiento permite disponer de todas las herramientas requeridas para integrar OpenCV con Java y ejecutar posteriormente ejemplos prácticos relacionados con la manipulación y análisis de imágenes digitales.

a. Descarga desde el sitio oficial

Se accedió al sitio oficial <https://opencv.org/releases/> donde se listan todas las versiones disponibles. Se seleccionó OpenCV 4.10.0 (2024-06-03) por ser la más reciente con soporte Java estable, haciendo clic en el botón Windows.

Releases

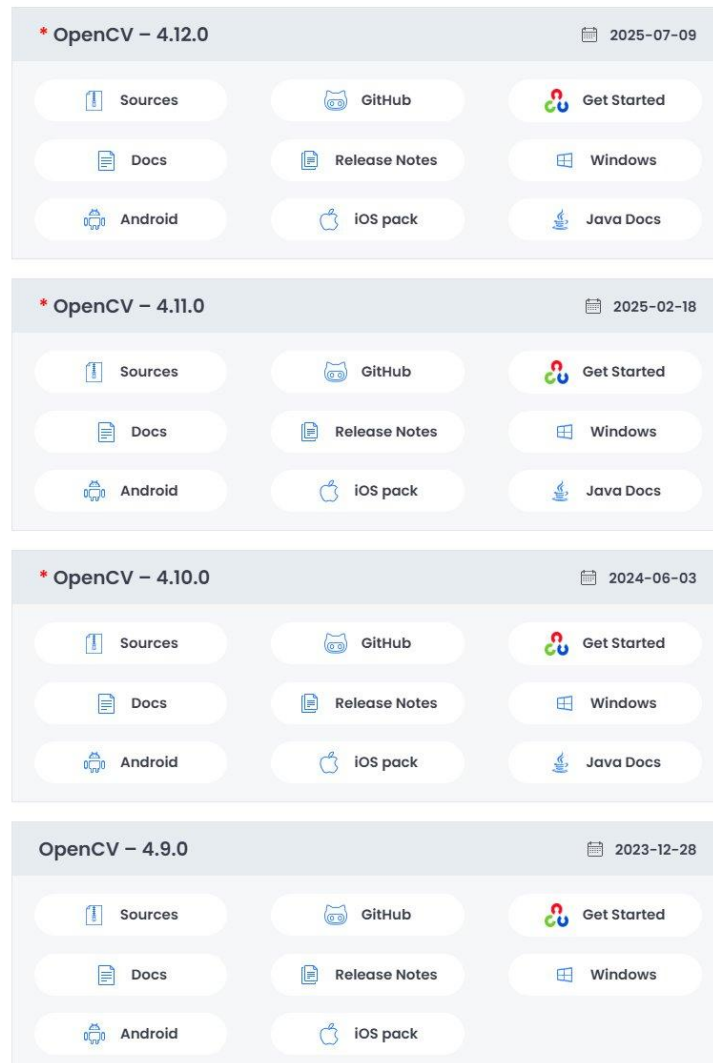


Figura 1. Página de releases de OpenCV - versiones disponibles.

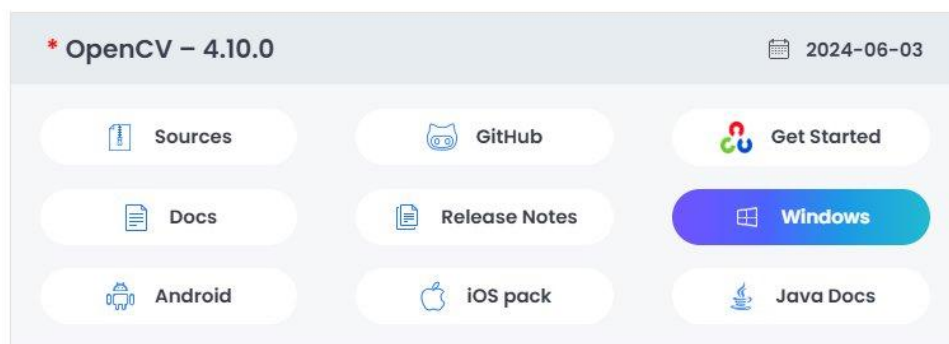


Figura 2. Selección de OpenCV 4.10.0 para Windows.

b. Extracción del archivo

El instalador descargado es un archivo autoextraíble de 7-Zip de 178 MB. Al ejecutarlo solicita la ruta de extracción. Se extrajo en la carpeta de Descargas y luego se movió a C:\opencv para mantener una ruta limpia.

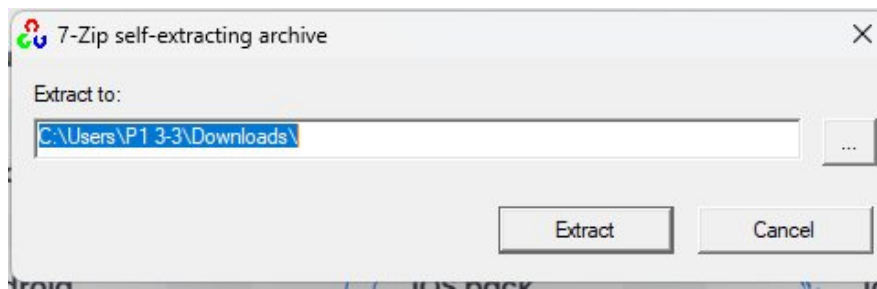


Figura 3. Diálogo de extracción del archivo autoextraíble.

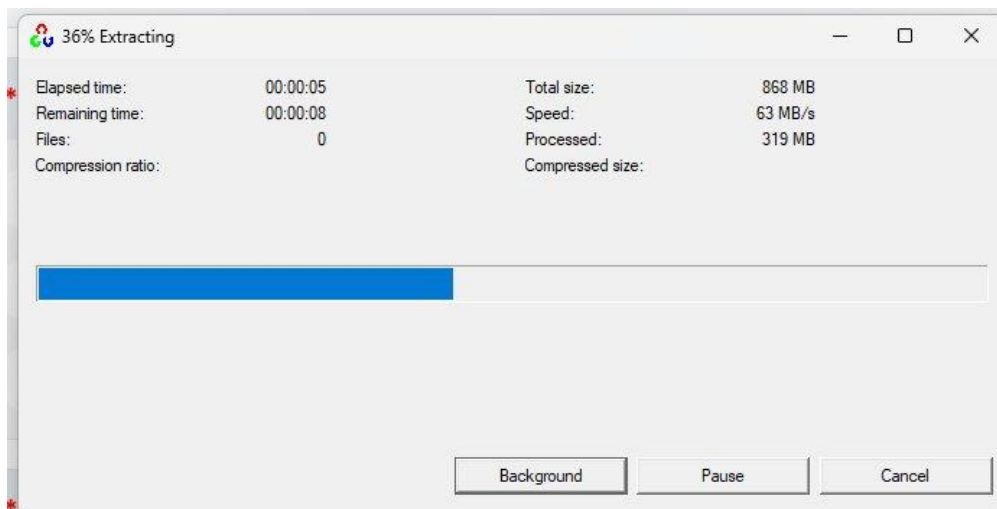


Figura 4. Progreso de extracción - 868 MB en total.

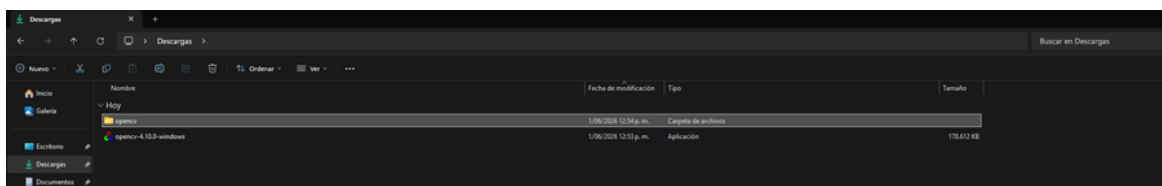


Figura 5. Carpeta Descargas con el instalador y la carpeta opencv extraída.

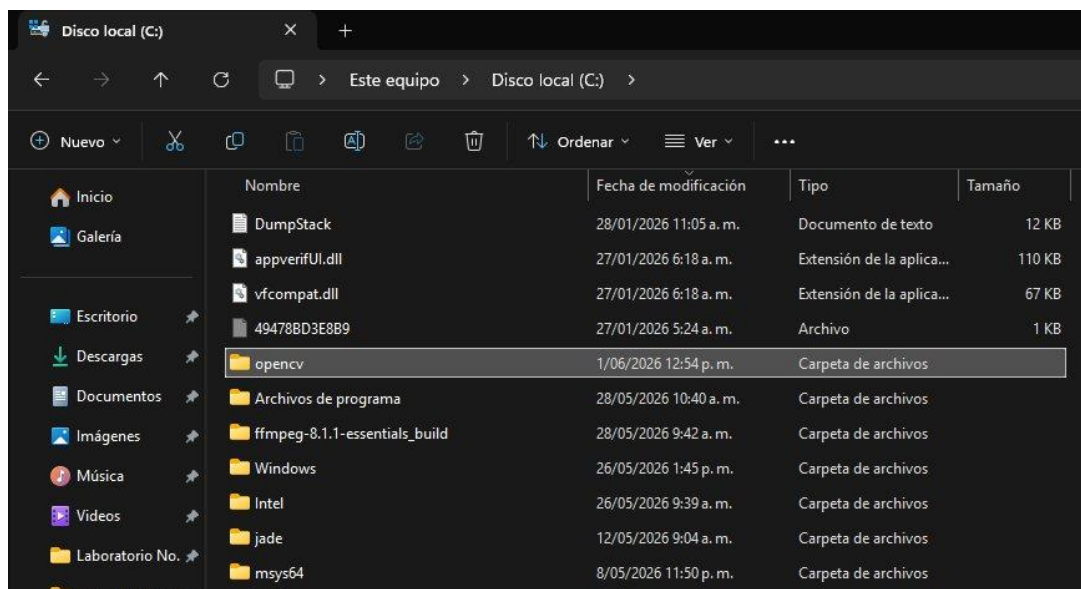


Figura 6. Carpeta opencv ubicada en la raíz de C:\.

c. Verificación de la estructura de archivos

Se navegó hasta C:\opencv\build\java\ para confirmar la presencia de los archivos necesarios para la integración con Java: el archivo JAR y las carpetas con las DLL nativas para 32 y 64 bits.

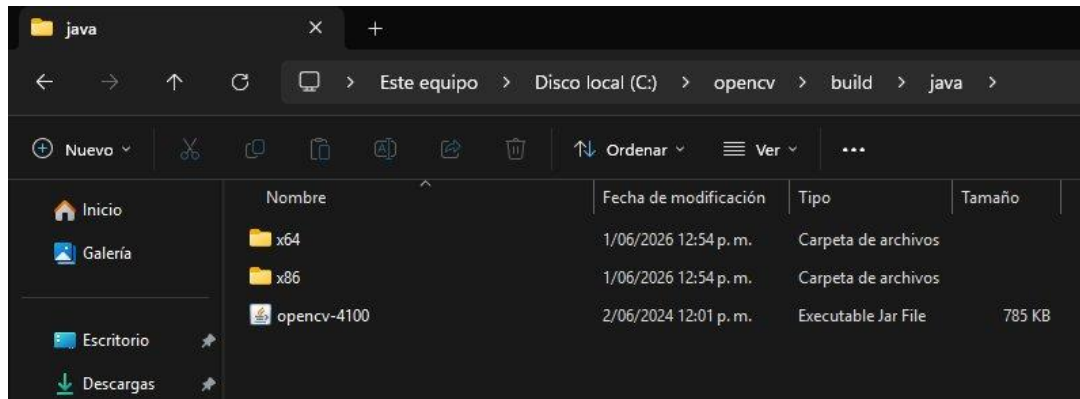


Figura 7. Contenido de C:\opencv\build\java\ - JAR y carpetas x64/x86.

Archivos clave para integración Java:

- JAR: C:\opencv\build\java\opencv-4100.jar (785 KB)
- DLL x64: C:\opencv\build\java\x64\opencv_java4100.dll
- Cascades: C:\opencv\build\etc\haarcascades\

3. Configuración del Proyecto en NetBeans

Una vez instalada la biblioteca OpenCV, es necesario configurar correctamente el entorno de desarrollo para permitir su integración con aplicaciones Java. En esta etapa se realiza la creación del proyecto en Apache NetBeans, la incorporación de las librerías requeridas y la definición de las rutas necesarias para el acceso a los archivos nativos de OpenCV. Esta configuración garantiza que el entorno pueda reconocer y utilizar las funcionalidades de la biblioteca durante la compilación y ejecución de los programas desarrollados.

a. Creación del proyecto

Se abrió Apache NetBeans IDE 20 y se creó un nuevo proyecto mediante File → New Project. Se seleccionó Java with Ant → Java Application. El proyecto se denominó LaboratorioOpenCV.

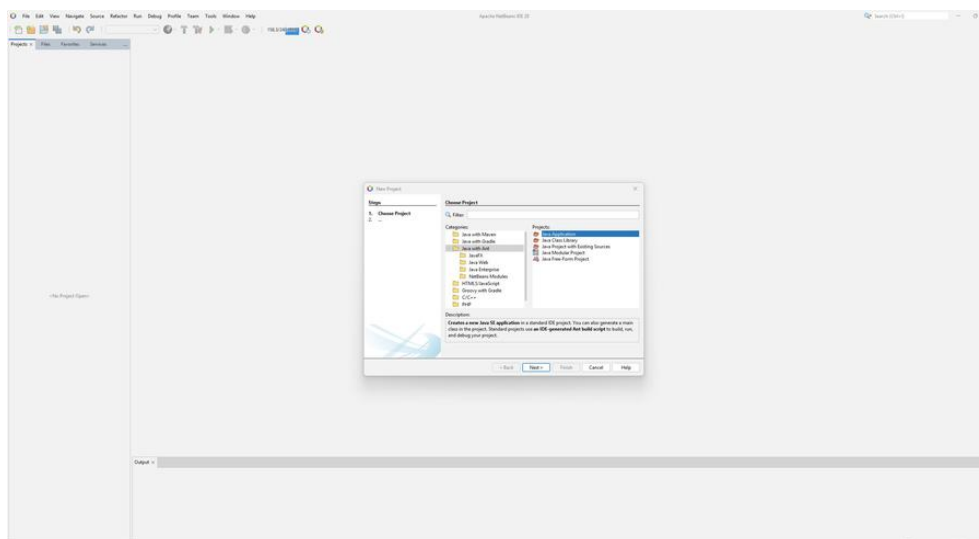


Figura 8. Asistente de nuevo proyecto - selección de Java Application.

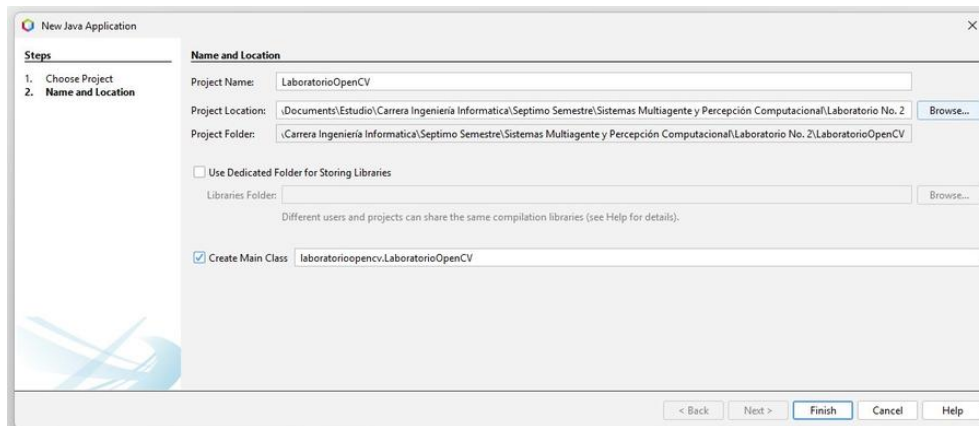


Figura 9. Configuración del nombre y ubicación del proyecto.

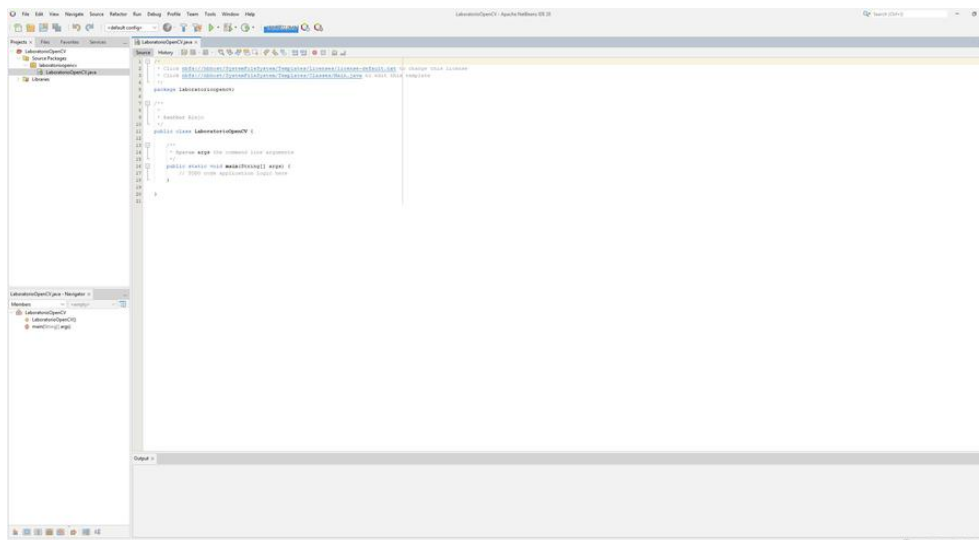


Figura 10. Proyecto LaboratorioOpenCV creado con clase principal generada.

b. Agregar la librería JAR

Para compilar con las clases de OpenCV se agregó el archivo JAR mediante clic derecho sobre Libraries → Add JAR/Folder, navegando hasta C:\opencv\build\java\.

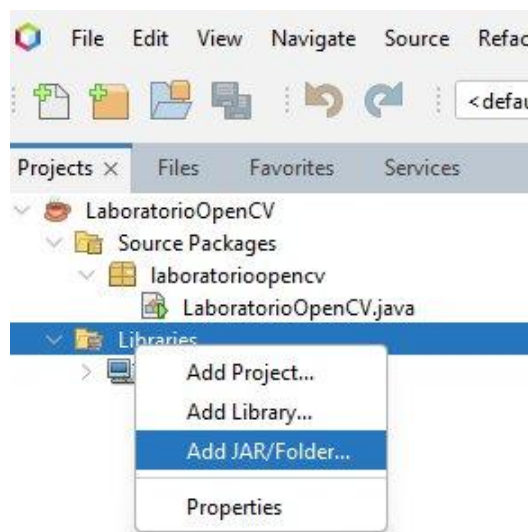


Figura 11. Menú contextual para agregar JAR al proyecto.

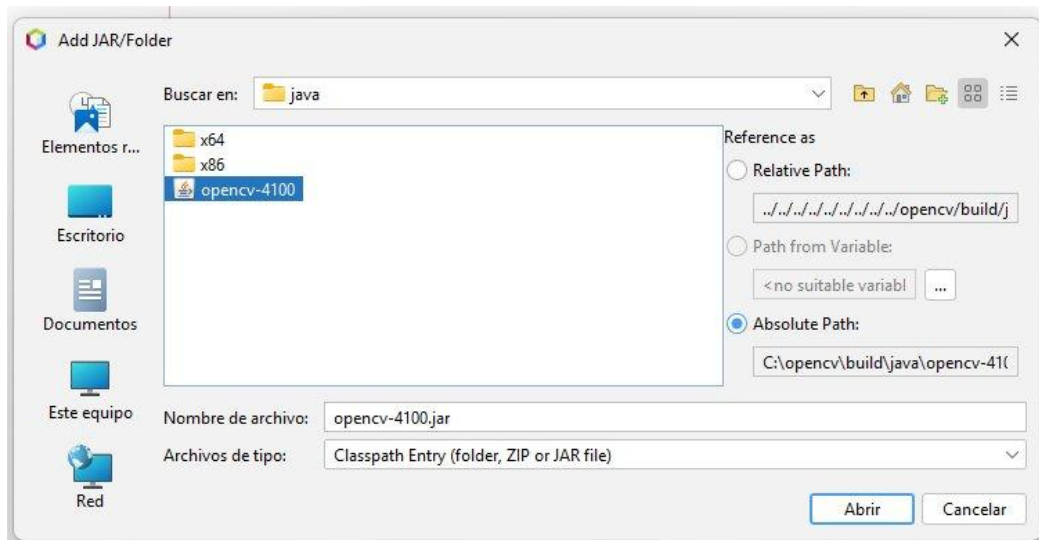


Figura 12. Diálogo de selección del archivo opencv-4100.jar.

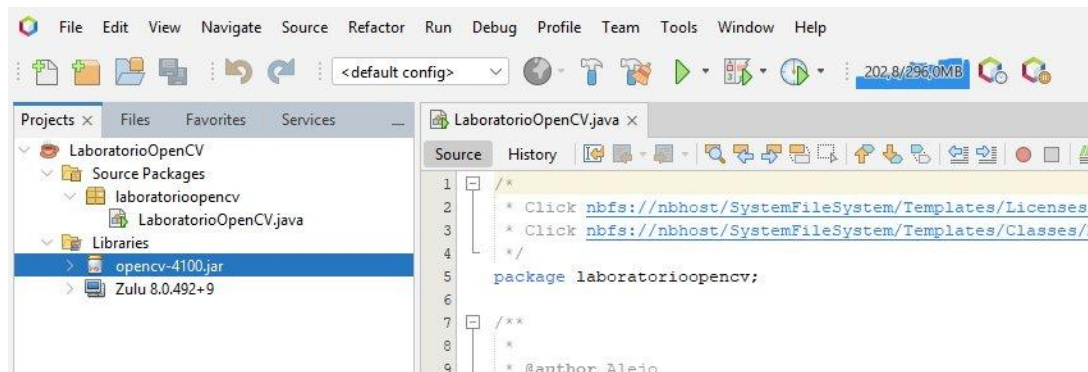


Figura 13. Librería opencv-4100.jar agregada correctamente al proyecto.

c. Configuración de la VM Option

OpenCV requiere que la JVM encuentre la librería nativa (.dll) en tiempo de ejecución. Se configura mediante `-Djava.library.path` en Properties → Run → VM Options.

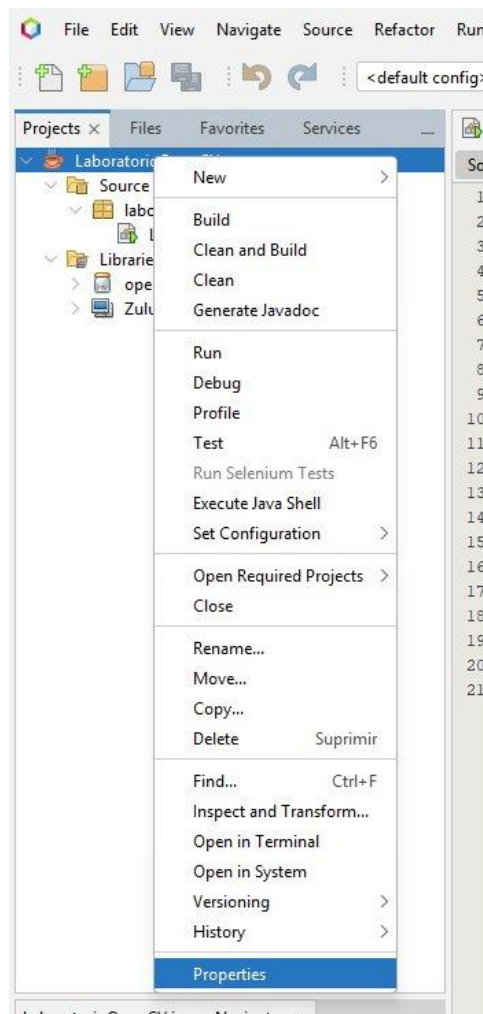


Figura 14. Acceso a *Properties* del proyecto.

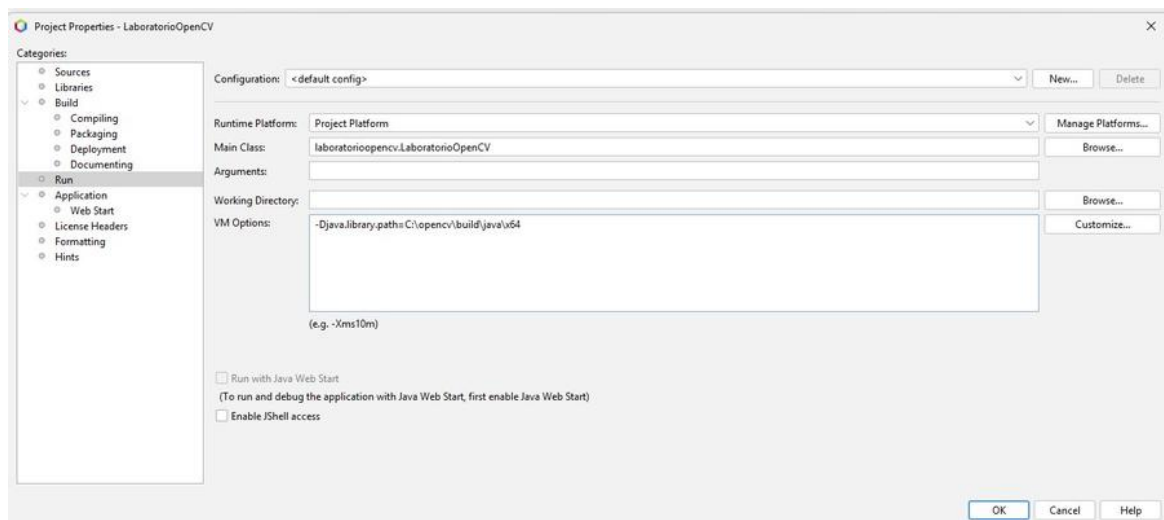


Figura 15. VM Option: `-Djava.library.path=C:\opencv\build\x64`

d. Selección de la plataforma Java

Al ejecutar la primera prueba se presentó el error `UnsupportedClassVersionError: OpenCV 4.10.0 requiere Java 11 (class file 55.0) pero el proyecto usaba Java 8 (class file 52.0)`. Se solucionó seleccionando Zulu 17.0.19+10 en *Properties* → *Libraries* → *Java Platform*.

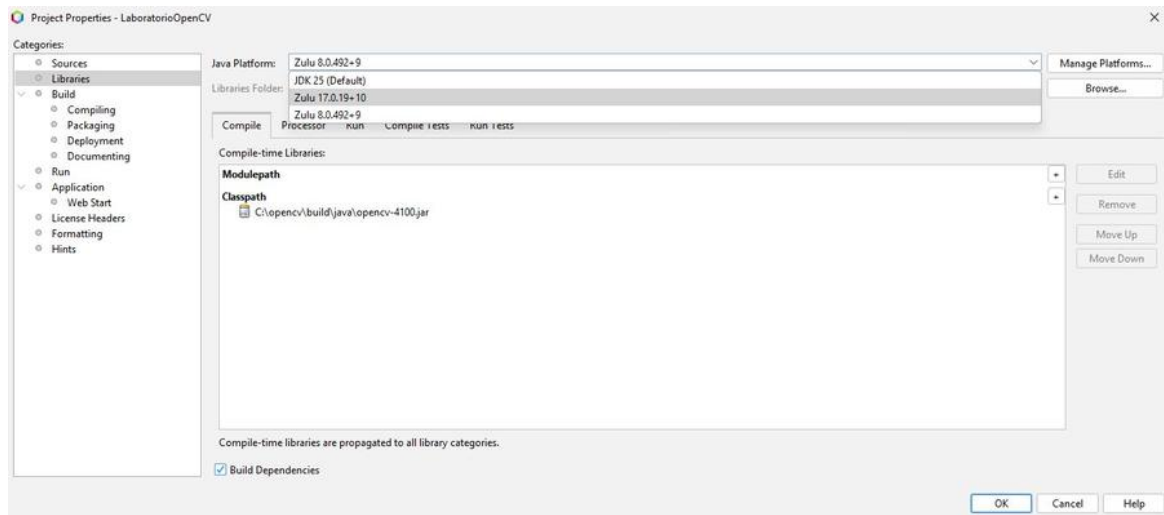


Figura 16. Error de incompatibilidad de versión Java al usar JDK 8.

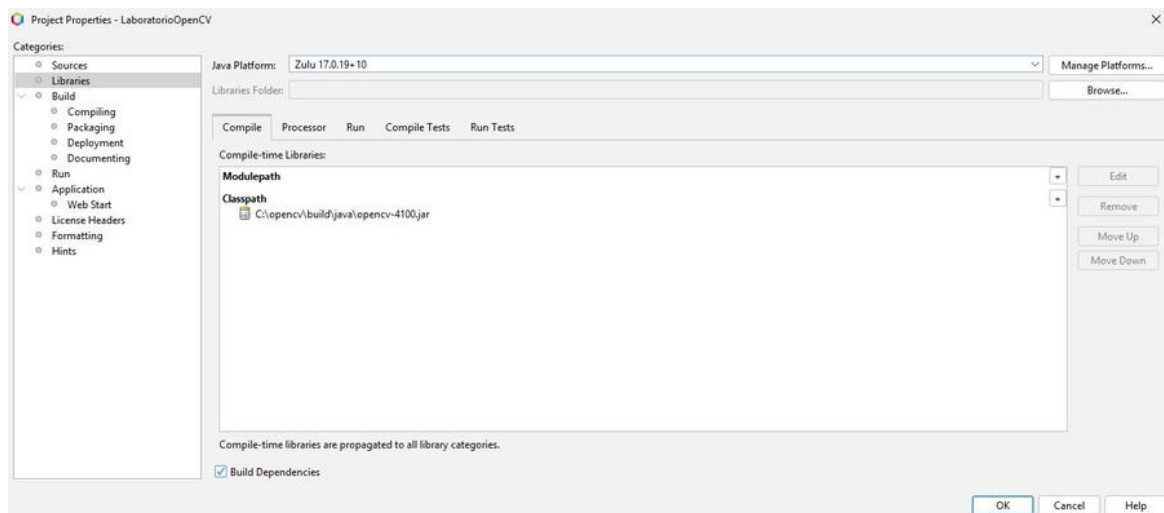


Figura 17. Selección de Zulu 17.0.19+10 como plataforma del proyecto.

e. Verificación de la instalación

Se implementó un programa de prueba que carga la librería nativa e imprime la versión de OpenCV. La ejecución exitosa confirma la configuración correcta.

```
System.loadLibrary(Core.NATIVE_LIBRARY_NAME);
System.out.println("OpenCV version: " + Core.VERSION);
```

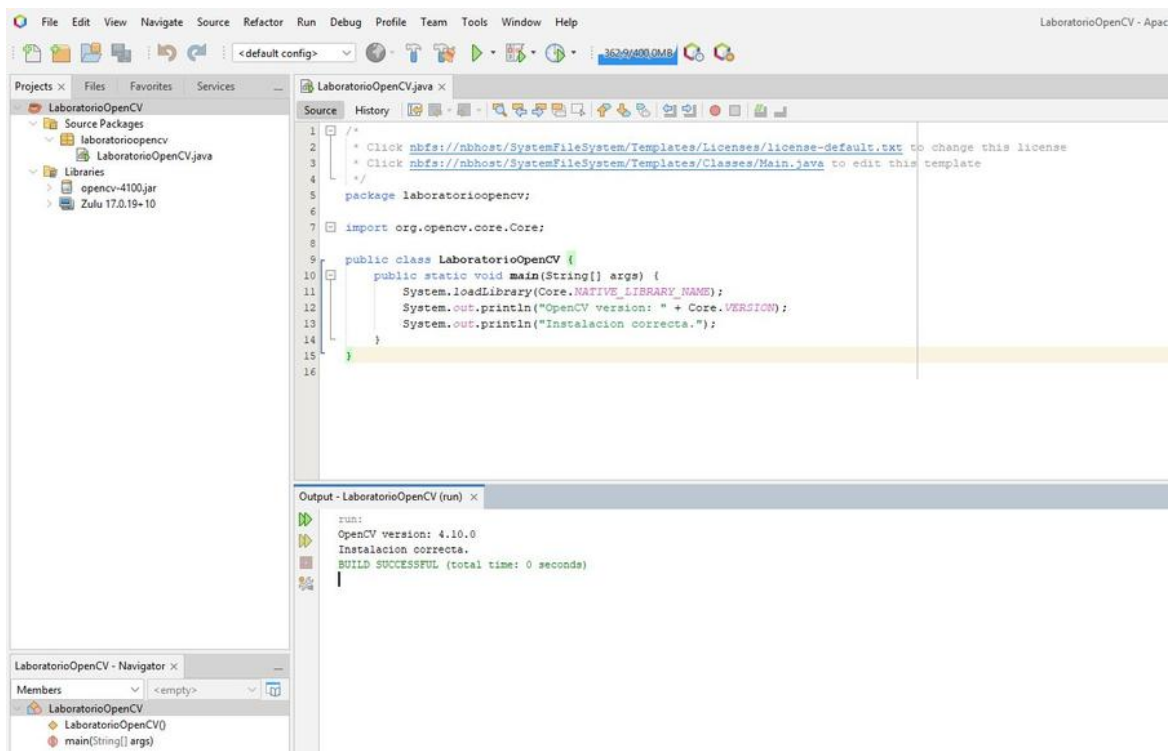


Figura 18. Verificación exitosa: OpenCV version 4.10.0 - BUILD SUCCESSFUL.

4. Rutinas de Visión Artificial

Se implementaron cuatro clases Java independientes, cada una demostrando una rutina diferente. Cada clase incluye modificaciones al código base de los tutoriales oficiales de OpenCV.

a. Rutina 1 - Procesamiento Básico (Rutina1Basico.java)

Demuestra las operaciones fundamentales: conversión a escala de grises, filtro gaussiano para reducir ruido, y umbralización binaria para segmentar la imagen. La modificación al tutorial consistió en agregar la umbralización binaria como paso adicional a la conversión básica a grises.

i. Explicación del Código – Rutina 1 Básica Procesamiento Imágenes OpenCV

Este programa tiene como objetivo demostrar algunas de las operaciones fundamentales de procesamiento digital de imágenes utilizando la biblioteca OpenCV en Java. A través de una imagen de entrada, se aplican diferentes transformaciones que permiten obtener nuevas versiones de la imagen para su posterior análisis.

Inicialmente, el programa importa las clases necesarias de OpenCV para trabajar con matrices de imágenes ('Mat'), operaciones de procesamiento ('Imgproc'), lectura y escritura de archivos gráficos ('Imgcodecs') y acceso a las librerías nativas del framework ('Core').

Posteriormente, se define una variable denominada 'ruta', la cual almacena la ubicación donde se encuentran las imágenes utilizadas durante el laboratorio. Esta ruta será empleada tanto para cargar la imagen original como para guardar los resultados generados.

Dentro del método 'main()', el programa comienza mostrando la ruta de trabajo y cargando la librería nativa de OpenCV mediante la instrucción:

```
System.loadLibrary(Core.NATIVE_LIBRARY_NAME);
```

Esta línea es indispensable, ya que permite que Java tenga acceso a las funciones implementadas internamente por OpenCV.

A continuación, se carga la imagen original utilizando el método `imread()`:

```
Mat imgOriginal = Imgcodecs.imread(ruta + "foto1.png");
```

La imagen es almacenada en un objeto de tipo `Mat`, estructura fundamental de OpenCV que representa una matriz de píxeles.

Seguidamente, se realiza una validación para verificar que la imagen haya sido cargada correctamente:

```
if (imgOriginal.empty())
```

Si la imagen no existe o no puede ser leída, el programa muestra un mensaje de error y finaliza su ejecución.

La primera operación de procesamiento corresponde a la conversión de la imagen a escala de grises:

```
Imgproc.cvtColor(imgOriginal, imgGris, Imgproc.COLOR_BGR2GRAY);
```

Esta transformación elimina la información de color y conserva únicamente los niveles de intensidad luminosa, simplificando el análisis posterior de la imagen.

Posteriormente, se aplica un filtro Gaussiano:

```
Imgproc.GaussianBlur(imgGris, imgSuavizada, new Size(15, 15), 0);
```

El objetivo de este filtro es reducir el ruido presente en la imagen y suavizar las transiciones entre píxeles. El tamaño del kernel utilizado es de 15×15 píxeles, permitiendo obtener una imagen más uniforme.

Luego se realiza una umbralización binaria mediante la función:

```
Imgproc.threshold(imgGris, imgBinaria, 127, 255, Imgproc.THRESH_BINARY);
```

Esta técnica convierte la imagen en blanco y negro. Los píxeles con intensidad superior a 127 toman el valor máximo (255), mientras que los inferiores se convierten en negro (0). Este procedimiento es ampliamente utilizado para la segmentación de objetos y la detección de formas.

Una vez generadas las imágenes procesadas, el programa las almacena en disco mediante:

```
Imgcodecs.imwrite(...)
```

Se crean tres nuevos archivos:

- * `foto1_gris.png`: Imagen en escala de grises.
- * `foto1_suavizada.png`: Imagen con filtro Gaussiano aplicado.
- * `foto1_binaria.png`: Imagen binarizada mediante umbralización.

Finalmente, el programa informa que la rutina fue ejecutada correctamente y muestra las dimensiones de la imagen original:

```
imgOriginal.rows()  
imgOriginal.cols()
```

Estos valores corresponden al número de filas (alto) y columnas (ancho) de la imagen cargada.

En conclusión, esta rutina permite comprender tres operaciones esenciales del procesamiento digital de imágenes: conversión a escala de grises, reducción de ruido mediante suavizado Gaussiano y segmentación por umbralización binaria. Estas técnicas constituyen la base de numerosos algoritmos de visión artificial utilizados en aplicaciones de reconocimiento de objetos, detección de patrones y análisis automatizado de imágenes.

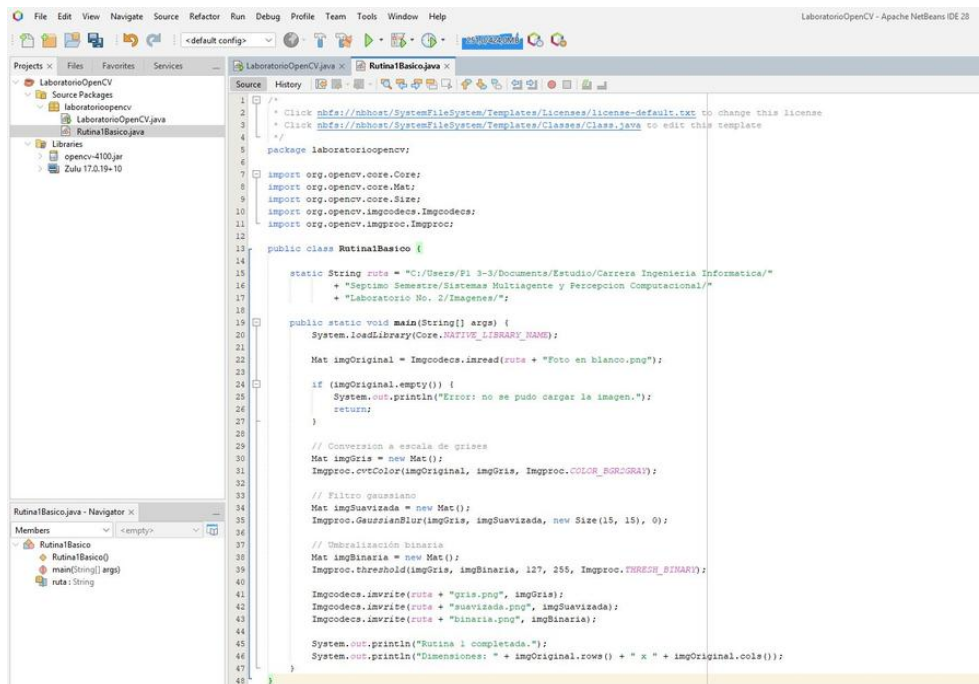


Figura 19. Código de Rutina1Basico.java en NetBeans IDE 28.

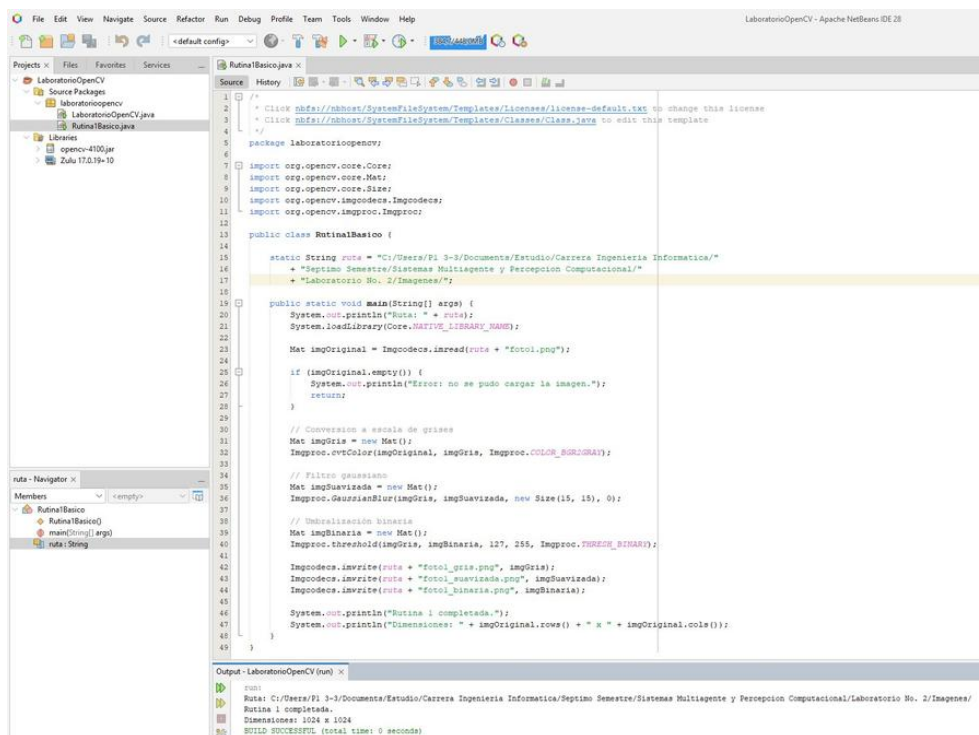


Figura 20. Ejecución exitosa - Dimensiones 1024x1024 - BUILD SUCCESSFUL.

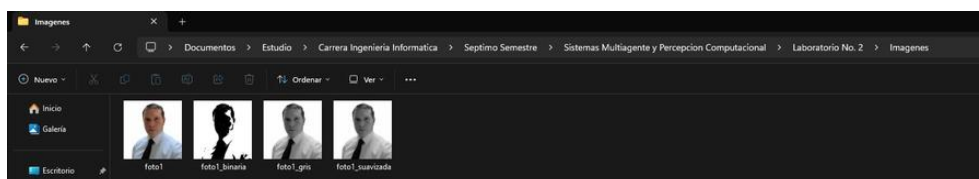


Figura 21. Resultados: imagen original, binaria, gris y suavizada.



Figura 22. Resultados: imagen original



Figura 23. Resultados: imagen binaria



Figura 24. Resultados: imagen gris



Figura 25. Resultados: imagen suavizada

b. Rutina 2 - Detección de Bordes (Rutina2Bordes.java)

Se implementaron dos operadores clásicos de detección de bordes: Canny y Sobel. La modificación consistió en implementar ambos en la misma clase y combinar los gradientes Sobel X e Y mediante `Core.addWeighted()` para obtener el gradiente total.

i. Explicación del Código – Rutina 2: Detección de Bordes con Canny y Sobel

Esta rutina tiene como objetivo aplicar técnicas de detección de bordes sobre una imagen digital utilizando la biblioteca OpenCV. La detección de bordes es una de las etapas más importantes dentro de la visión artificial, ya que permite identificar los contornos y límites de los objetos presentes en una

imagen, facilitando tareas posteriores como segmentación, reconocimiento de patrones y análisis de formas.

Inicialmente, el programa importa las clases necesarias de OpenCV para trabajar con imágenes, matrices y algoritmos de procesamiento digital. Entre ellas se encuentran las clases `Mat`, utilizadas para almacenar imágenes en forma de matrices, y las librerías `Imgproc` y `Core`, que proporcionan las funciones de procesamiento y análisis.

Al igual que en la rutina anterior, se define una variable denominada `ruta`, la cual contiene la ubicación donde se encuentra almacenada la imagen que será procesada.

Dentro del método principal `main()`, se carga la librería nativa de OpenCV mediante la instrucción:

```
System.loadLibrary(Core.NATIVE_LIBRARY_NAME);
```

Esta operación es obligatoria para que Java pueda acceder a las funciones internas implementadas por OpenCV.

Posteriormente, se carga la imagen original utilizando:

```
Mat imgOriginal = Imgcodecs.imread(ruta + "foto1.png");
```

La imagen es almacenada en una matriz de tipo `Mat`, estructura fundamental utilizada por OpenCV para representar imágenes digitales.

Antes de continuar con el procesamiento, se verifica que la imagen haya sido cargada correctamente:

```
if (imgOriginal.empty())
```

Si el archivo no existe o presenta algún problema de lectura, el programa muestra un mensaje de error y finaliza su ejecución.

Como paso previo a la detección de bordes, la imagen es convertida a escala de grises:

```
Imgproc.cvtColor(imgOriginal, imgGris, Imgproc.COLOR_BGR2GRAY);
```

Esta transformación simplifica el análisis, ya que elimina la información de color y conserva únicamente los niveles de intensidad de cada píxel.

La primera técnica utilizada para detectar bordes es el algoritmo de Canny:

```
Imgproc.Canny(imgGris, imgCanny, 80, 200);
```

El detector Canny identifica cambios bruscos de intensidad en la imagen, los cuales generalmente corresponden a los contornos de los objetos. Los valores 80 y 200 representan los umbrales inferior y superior utilizados para determinar qué píxeles forman parte de un borde válido.

A continuación, se aplica el operador Sobel en la dirección horizontal (eje X):

```
Imgproc.Sobel(imgGris, imgSobelX, CvType.CV_16S, 1, 0);
```

Este filtro calcula las variaciones de intensidad a lo largo de las columnas de la imagen, permitiendo detectar bordes verticales. Posteriormente, se utiliza:

```
Core.convertScaleAbs(imgSobelX, imgSobelX);
```

para convertir los resultados obtenidos a valores absolutos y hacerlos visibles como una imagen estándar.

De forma similar, se aplica el operador Sobel en la dirección vertical (eje Y):

```
Imgproc.Sobel(imgGris, imgSobelY, CvType.CV_16S, 0, 1);
```

En este caso, se detectan los cambios de intensidad en las filas de la imagen, resaltando principalmente los bordes horizontales.

Una vez obtenidos ambos resultados, se combinan mediante:

```
Core.addWeighted(imgSobelX, 0.5, imgSobelY, 0.5, 0, imgSobelTotal);
```

Esta operación genera una imagen final que integra la información de los bordes detectados tanto en dirección horizontal como vertical, proporcionando una representación más completa de los contornos presentes en la imagen.

Finalmente, los resultados son almacenados en disco utilizando:

```
Imgcodecs.imwrite(...)
```

Generando los siguientes archivos:

- `foto1\_canny.png`: Imagen con bordes detectados mediante el algoritmo de Canny.
- `foto1\_sobel.png`: Imagen con bordes detectados mediante la combinación de los operadores Sobel X y Sobel Y.

Al concluir el procesamiento, el programa muestra mensajes informativos indicando que la rutina fue ejecutada correctamente y que los bordes fueron detectados utilizando ambas técnicas.

En conclusión, esta rutina introduce dos de los métodos más utilizados en visión artificial para la detección de contornos. El algoritmo de Canny proporciona una detección precisa y robusta de bordes, mientras que el operador Sobel permite analizar las variaciones de intensidad en diferentes direcciones. Ambas técnicas constituyen herramientas fundamentales para aplicaciones de reconocimiento de objetos, segmentación de imágenes, inspección industrial automatizada y sistemas inteligentes basados en percepción computacional.

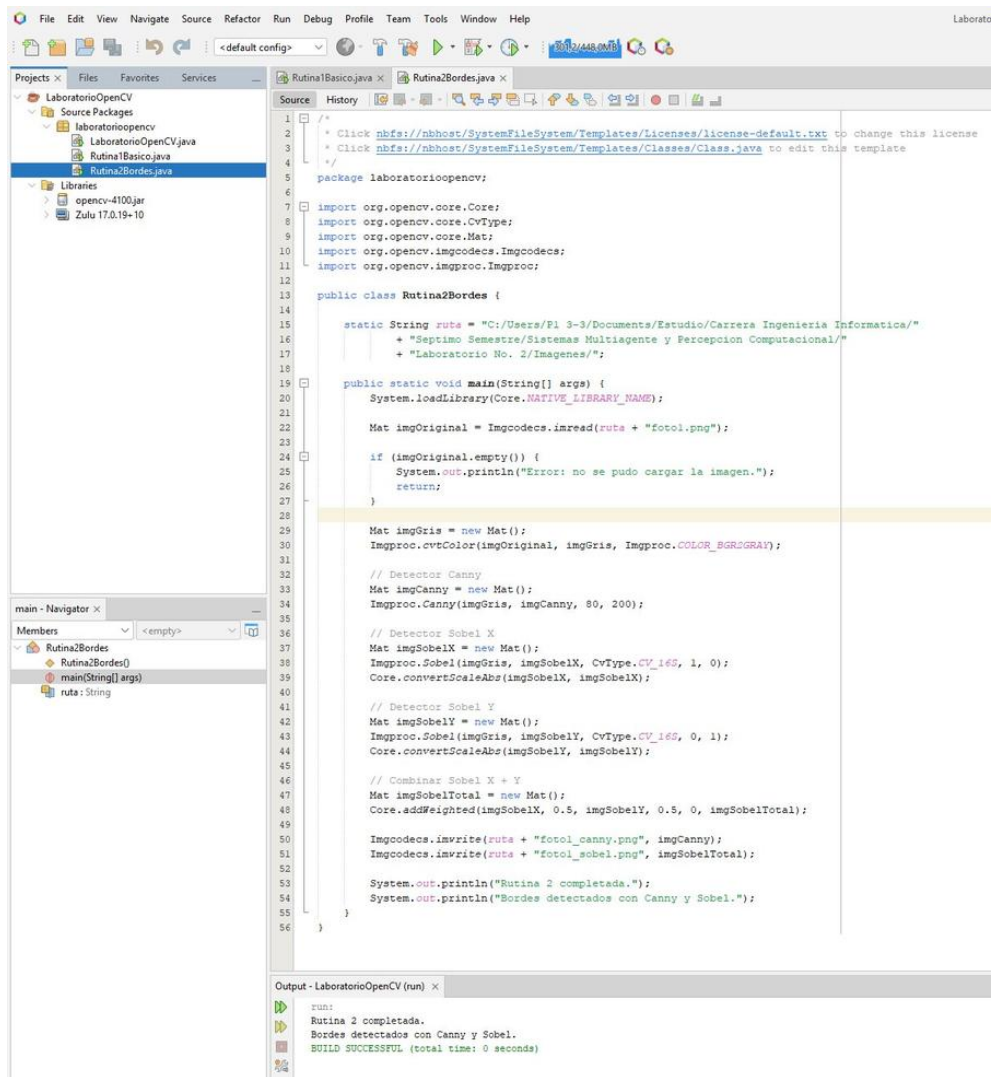


Figura 26. Código de Rutina2Bordes.java con operadores Canny y Sobel.

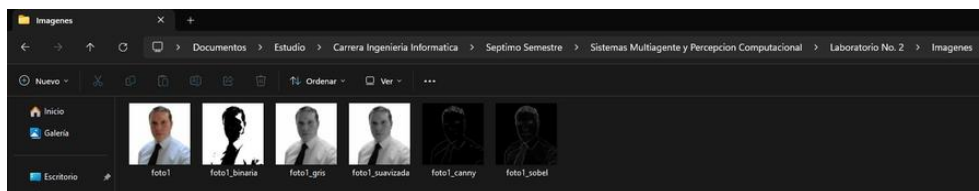


Figura 27. Resultados: Canny (izq.) y Sobel combinado (der.).



Figura 28. Resultados: imagen Canny

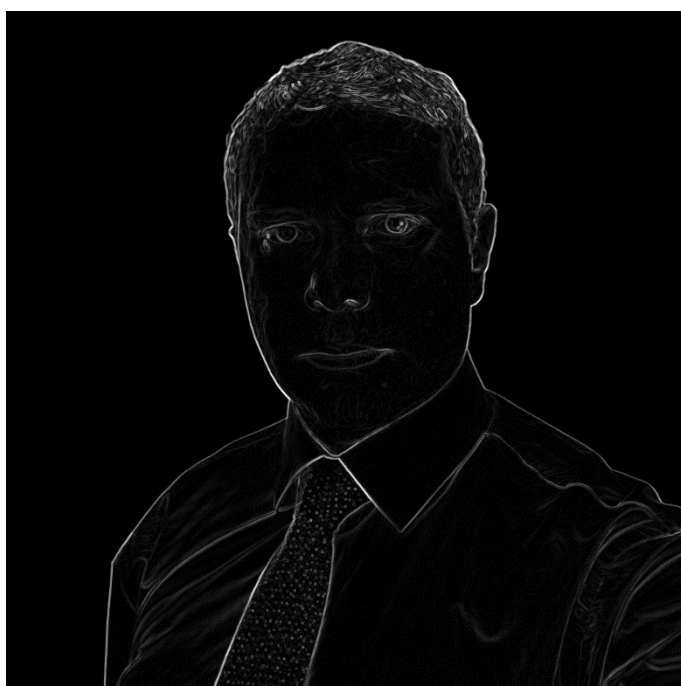


Figura 29. Resultados: imagen Sobel combinado

c. Rutina 3 - Detección de Rostros (Rutina3Rostros.java)

Se utilizó el clasificador Haar Cascade preentrenado para detectar rostros. Las modificaciones fueron: ecualización del histograma para mejorar la detección con iluminación irregular, ajuste de parámetros (scaleFactor=1.05, minNeighbors=2) para mayor sensibilidad, y etiquetado con putText() sobre cada rostro detectado.

i. Explicación del Código – Rutina 3: Detección de Rostros con Haar Cascade

Esta rutina tiene como objetivo implementar un sistema básico de detección de rostros utilizando la biblioteca OpenCV. Para ello se emplea un clasificador preentrenado basado en la técnica Haar

Cascade, uno de los métodos clásicos más utilizados en visión artificial para identificar automáticamente rostros humanos dentro de imágenes digitales.

Inicialmente, el programa importa las librerías necesarias de OpenCV para trabajar con imágenes, procesamiento digital y detección de objetos. Entre ellas destacan las clases `CascadeClassifier`, utilizada para cargar el modelo de detección facial, y las clases `Mat`, `Rect` y `MatOfRect`, encargadas de almacenar las imágenes y las regiones detectadas.

Al comienzo de la clase se definen dos rutas importantes:

```
static String ruta = "../Imagenes/";
```

La primera corresponde a la ubicación donde se encuentran las imágenes que serán procesadas durante el laboratorio.

```
static String rutaClasif = "C:/opencv/build/etc/haarcascades/"  
+ "haarcascade_frontalface_default.xml";
```

La segunda ruta apunta al archivo XML que contiene el modelo Haar Cascade entrenado para detectar rostros frontales.

Dentro del método principal `main()`, se carga la librería nativa de OpenCV mediante la instrucción:

```
System.loadLibrary(Core.NATIVE_LIBRARY_NAME);
```

Este paso es indispensable para que Java pueda acceder a las funciones implementadas por OpenCV.

Posteriormente, se crea el detector facial:

```
CascadeClassifier detector = new CascadeClassifier(rutaClasif);
```

Este objeto carga el clasificador Haar Cascade y será responsable de analizar la imagen en busca de rostros.

A continuación, se carga la imagen que será analizada:

```
Mat img = Imgcodecs.imread(ruta + "foto2.jpg");
```

La imagen es almacenada en una matriz de tipo `Mat`, estructura fundamental utilizada por OpenCV para representar imágenes digitales.

Antes de iniciar el procesamiento, se verifica que la imagen haya sido cargada correctamente:

```
if (img.empty())
```

Si la imagen no existe o no puede leerse, el programa muestra un mensaje de error y finaliza la ejecución.

Como etapa de preprocesamiento, la imagen es convertida a escala de grises:

```
Imgproc.cvtColor(img, imgGris, Imgproc.COLOR_BGR2GRAY);
```

La detección de rostros resulta más eficiente sobre imágenes en escala de grises, ya que se reduce la cantidad de información que debe analizar el algoritmo.

Posteriormente se mejora el contraste mediante ecualización del histograma:

```
Imgproc.equalizeHist(imgGris, imgGris);
```

Esta operación ayuda a resaltar las características faciales y mejora la precisión del detector en condiciones de iluminación variables.

La detección de rostros se realiza mediante el método:

```
detector.detectMultiScale(  
    imgGris,  
    rostros,  
    1.05,  
    2,  
    0,  
    new Size(20, 20),  
    new Size());
```

Este algoritmo analiza la imagen en diferentes escalas y busca patrones compatibles con rostros humanos. Los rostros encontrados son almacenados dentro de la variable `rostros`, que contiene una colección de rectángulos con las coordenadas de cada detección.

Una vez finalizada la búsqueda, el programa muestra en consola la cantidad total de rostros encontrados:

```
System.out.println("Rostros detectados: "  
    + rostros.toArray().length);
```

Posteriormente, se recorre cada uno de los rostros detectados:

```
for (Rect r : rostros.toArray())
```

Para cada detección se dibuja un rectángulo verde alrededor del rostro:

```
Imgproc.rectangle(  
    img,  
    new Point(r.x, r.y),  
    new Point(r.x + r.width, r.y + r.height),  
    new Scalar(0, 255, 0),  
    3);
```

Este rectángulo permite visualizar claramente la región donde el sistema identificó una cara.

Además, se agrega una etiqueta descriptiva:

```
Imgproc.putText(  
    img,  
    "Rostro",  
    new Point(r.x, r.y - 10),  
    Imgproc.FONT_HERSHEY_SIMPLEX,  
    0.8,  
    new Scalar(0, 255, 0),  
    2);
```

La palabra "Rostro" aparece sobre cada detección, facilitando la interpretación visual de los resultados.

Finalmente, la imagen procesada es almacenada en disco mediante:

```
Imgcodecs.imwrite(  
    ruta + "foto2_rostros.png",  
    img);
```

Esto genera una nueva imagen que conserva la original, pero incorporando los rectángulos y etiquetas correspondientes a los rostros detectados.

Al concluir la ejecución, el programa muestra un mensaje indicando que la rutina fue completada correctamente:

```
System.out.println("Rutina 3 completada.");
```

En conclusión, esta rutina demuestra la aplicación práctica de técnicas de percepción computacional para la detección automática de rostros utilizando clasificadores Haar Cascade. Este tipo de algoritmos constituye la base de numerosos sistemas de reconocimiento facial, control de acceso biométrico, videovigilancia inteligente, interacción humano-computador y aplicaciones modernas de inteligencia artificial basadas en visión por computador.

```

4 package laboratorioopencv;
5
6
7 import org.opencv.core.Core;
8 import org.opencv.core.Mat;
9 import org.opencv.core.MatOfRect;
10 import org.opencv.core.Point;
11 import org.opencv.core.Rect;
12 import org.opencv.core.Scalar;
13 import org.opencv.core.Size;
14 import org.opencv.imgcodecs.Imgcodecs;
15 import org.opencv.imgproc.Imgproc;
16 import org.opencv.objdetect.CascadeClassifier;
17
18 public class Rutina3Rostros {
19
20     static String ruta = "C:/Users/Pl 3-3/Documents/Estudio/Carrera Ingenieria Informatica/"
21         + "Septimo Semestre/Sistemas Multiagente y Percepcion Computacional/"
22         + "Laboratorio No. 2/Imagenes/";
23
24     static String rutaClasif = "C:/opencv/build/etc/haarcascades/"
25         + "haarcascade_frontalface_default.xml";
26
27     public static void main(String[] args) {
28         System.loadLibrary(Core.NATIVE_LIBRARY_NAME);
29
30         CascadeClassifier detector = new CascadeClassifier(rutaClasif);
31
32         Mat img = Imgcodecs.imread(ruta + "foto2.jpg");
33
34         if (img.empty()) {
35             System.out.println("Error: no se pudo cargar la imagen.");
36             return;
37         }
38
39         Mat imgGris = new Mat();
40         Imgproc.cvtColor(img, imgGris, Imgproc.COLOR_BGR2GRAY);
41         Imgproc.equalizeHist(imgGris, imgGris);
42
43         MatOfRect rostros = new MatOfRect();
44         detector.detectMultiScale(imgGris, rostros, 1.05, 2,
45             0, new Size(20, 20), new Size());
46
47         System.out.println("Rostros detectados: " + rostros.toArray().length);
48
49         for (Rect r : rostros.toArray()) {
50             Imgproc.rectangle(img,
51                 new Point(r.x, r.y),
52                 new Point(r.x + r.width, r.y + r.height),
53                 new Scalar(0, 255, 0), 3);
54             Imgproc.putText(img, "Rostro",
55                 new Point(r.x, r.y - 10),
56                 Imgproc.FONT_HERSHEY_SIMPLEX,
57                 0.8, new Scalar(0, 255, 0), 2);
58         }
59
60         Imgcodecs.imwrite(ruta + "foto2_rostros.png", img);
61         System.out.println("Rutina 3 completada.");
62     }
63 }

```

Output - LaboratorioOpenCV (run) x

```

run:
Rostros detectados: 5
Rutina 3 completada.
BUILD SUCCESSFUL (total time: 1 second)

```

Figura 30. Código de Rutina3Rostros.java - 5 rostros detectados - BUILD SUCCESSFUL.

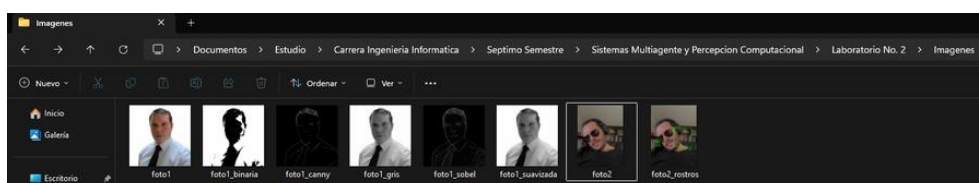


Figura 31. Resultado foto2_rostros.png con rectángulos verdes sobre rostros.

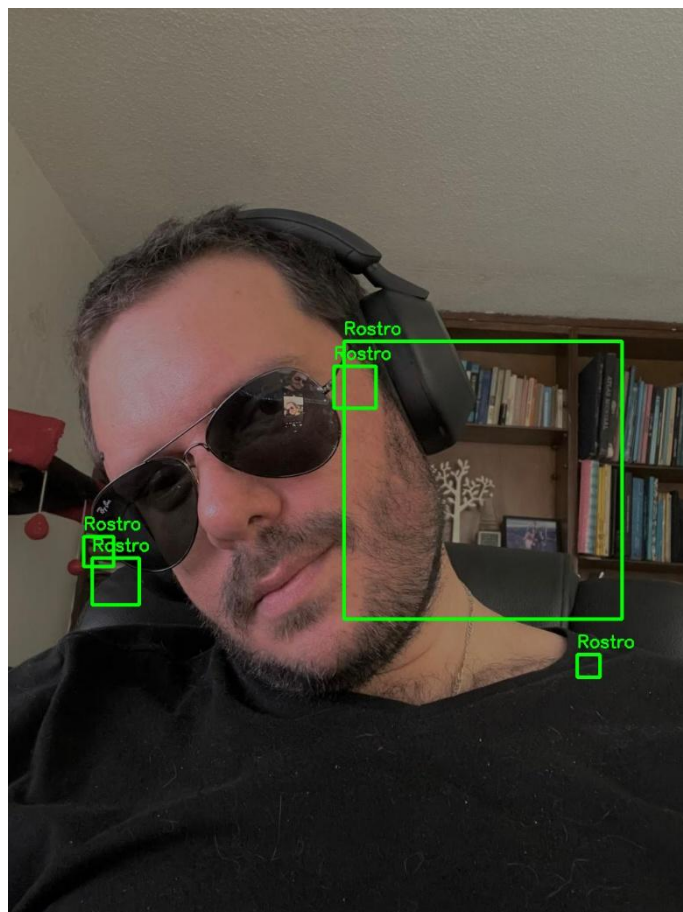


Figura 32. Resultados: imagen con rectángulos verdes sobre rostros.

d. Rutina 4 - Transformaciones Geométricas (Rutina4Transformaciones.java)

Se implementaron cuatro transformaciones: redimensionado al 50%, rotación 45 grados, volteo horizontal y volteo vertical. La modificación fue agregar el volteo vertical adicional e imprimir dimensiones antes y después para verificar la transformación.

i. Explicación del Código – Rutina 4: Transformaciones Geométricas Imágenes

Esta rutina tiene como objetivo aplicar diferentes transformaciones geométricas sobre una imagen digital utilizando las funcionalidades proporcionadas por OpenCV. Las transformaciones geométricas permiten modificar la posición, orientación y tamaño de los objetos presentes en una imagen sin alterar su contenido visual esencial. Este tipo de operaciones es ampliamente utilizado en aplicaciones de visión artificial, reconocimiento de patrones, análisis de imágenes y preparación de datos para algoritmos de inteligencia artificial.

Inicialmente, el programa importa las librerías necesarias de OpenCV para trabajar con imágenes, matrices y transformaciones geométricas. Entre ellas se encuentran las clases `Mat`, utilizadas para almacenar imágenes, y las clases `Imgproc` y `Core`, que proporcionan los métodos necesarios para realizar operaciones de procesamiento digital.

Al comienzo de la clase se define la variable:

```
static String ruta = ".../Imagenes/";
```

Esta variable almacena la ubicación donde se encuentra la imagen que será utilizada durante la ejecución de la rutina.

Dentro del método principal `main()`, se carga la librería nativa de OpenCV mediante la instrucción:

```
System.loadLibrary(Core.NATIVE_LIBRARY_NAME);
```

Este paso es obligatorio para habilitar el acceso a las funciones internas de OpenCV desde Java.

Posteriormente, se carga la imagen original utilizando:

```
Mat img = Imgcodecs.imread(ruta + "foto1.png");
```

La imagen es almacenada en una matriz de tipo Mat, estructura utilizada por OpenCV para representar imágenes digitales.

Antes de iniciar el procesamiento, se verifica que la imagen haya sido cargada correctamente:

```
if (img.empty())
```

Si la imagen no existe o presenta algún problema de lectura, el programa muestra un mensaje de error y finaliza su ejecución.

La primera transformación aplicada es el redimensionamiento de la imagen al 50 % de su tamaño original:

```
Imgproc.resize(  
    img,  
    imgRedim,  
    new Size(img.cols() * 0.5,  
             img.rows() * 0.5));
```

Esta operación reduce tanto el ancho como la altura de la imagen a la mitad, generando una nueva versión con menores dimensiones. El redimensionamiento es una técnica muy utilizada para optimizar el procesamiento de imágenes y disminuir el consumo de recursos computacionales.

La segunda transformación corresponde a una rotación de 45 grados. Para ello, primero se calcula el centro de la imagen:

```
Point centro = new Point(  
    img.cols() / 2.0,  
    img.rows() / 2.0);
```

Posteriormente se genera la matriz de transformación:

```
Mat matRot =  
    Imgproc.getRotationMatrix2D(  
        centro,  
        45,  
        1.0);
```

El valor 45 indica el ángulo de rotación en grados, mientras que 1.0 representa el factor de escala utilizado durante la transformación.

La rotación se aplica mediante:

```
Imgproc.warpAffine(  
    img,  
    imgRotada,  
    matRot,  
    new Size(  
        img.cols(),  
        img.rows()));
```

Esta operación genera una nueva imagen donde todos los píxeles han sido reposicionados según la matriz de rotación calculada previamente.

La tercera transformación consiste en realizar un volteo horizontal de la imagen:

```
Core.flip(img, imgEspejoH, 1);
```

El parámetro 1 indica que la imagen será reflejada respecto al eje vertical, produciendo un efecto espejo de izquierda a derecha.

Posteriormente se realiza un volteo vertical:

```
Core.flip(img, imgEspejoV, 0);
```

En este caso, el parámetro 0 indica que la reflexión se efectuará respecto al eje horizontal, generando una inversión de arriba hacia abajo.

Una vez realizadas todas las transformaciones, los resultados son almacenados en disco mediante:

```
Imgcodecs.imwrite(...)
```

Generando los siguientes archivos:

- foto1_redim.png: Imagen redimensionada al 50 % de su tamaño original.
- foto1_rotada.png: Imagen rotada 45 grados respecto a su centro.
- foto1_espejo_h.png: Imagen reflejada horizontalmente.
- foto1_espejo_v.png: Imagen reflejada verticalmente.

Finalmente, el programa muestra información sobre el proceso realizado:

```
System.out.println("Rutina 4 completada.");
```

Además, imprime las dimensiones de la imagen original y de la imagen redimensionada:

```
System.out.println("Tam. original: "
    + img.cols()
    + " x "
    + img.rows());

System.out.println("Tam. redimensionado: "
    + imgRedim.cols()
    + " x "
    + imgRedim.rows());
```

Esta información permite verificar que el cambio de tamaño se realizó correctamente.

En conclusión, esta rutina introduce algunas de las transformaciones geométricas más utilizadas en procesamiento digital de imágenes. Operaciones como el redimensionamiento, la rotación y los reflejos son fundamentales en aplicaciones de visión artificial, ya que permiten adaptar imágenes a diferentes condiciones de análisis, mejorar procesos de reconocimiento automático y preparar conjuntos de datos para sistemas avanzados de aprendizaje automático y percepción computacional.

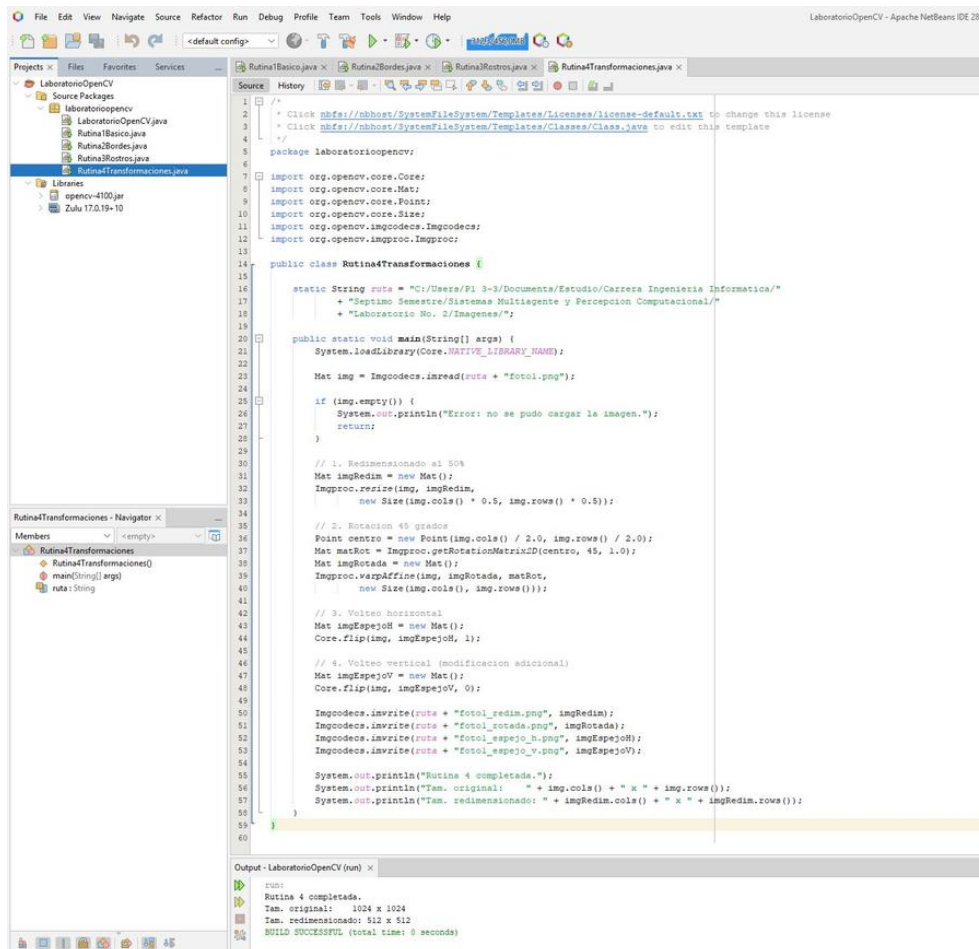


Figura 33. Código de Rutina4Transformaciones.java.

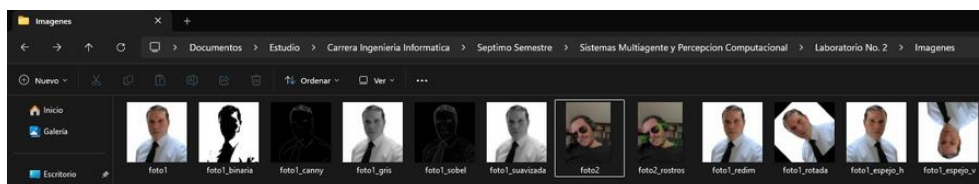


Figura 34. Resultados: redimensionada, rotada 45°, espejo horizontal y vertical.



Figura 35. Resultados: imagen redimensionada.



Figura 36. Resultados: imagen rotada 45°.



Figura 37. Resultados: imagen espejo horizontal.



Figura 38. Resultados: imagen espejo vertical.

5. Análisis de Resultados

A continuación, se presenta el resumen de las cuatro rutinas implementadas con sus modificaciones y resultados observados.

Rutina	Técnica base			Modificación aplicada		Resultado	
Rutina 1	Escala Gaussiano	grises	+	Umbralización adicional	binaria	Segmentación imagen 1024×1024	-

Rutina	Técnica base	Modificación aplicada	Resultado
Rutina 2	Canny (bordes)	Sobel X+Y fusionados, umbrales 80/200	Contornos con ambos operadores
Rutina 3	Haar Cascade básico	equalizeHist + parámetros ajustados + etiquetas	5 rostros detectados
Rutina 4	Resize + Rotación + Flip-H	Volteo vertical adicional + impresión de dims	1024×1024 → 512×512 - OK

Tabla 1. Resumen comparativo de rutinas implementadas.

CONCLUSIONES DEL LABORATORIO

El desarrollo del presente laboratorio permitió cumplir satisfactoriamente los objetivos planteados en la actividad, logrando la instalación, configuración y ejecución exitosa de OpenCV dentro de un entorno de desarrollo Java utilizando Apache NetBeans. A través de esta práctica fue posible aplicar de manera efectiva conceptos fundamentales relacionados con procesamiento digital de imágenes, visión artificial y percepción computacional, áreas que constituyen una base tecnológica esencial para el desarrollo de sistemas inteligentes capaces de interpretar información visual proveniente del entorno.

Durante la implementación del proyecto se realizó la instalación completa de OpenCV 4.10.0 y su integración con Java mediante la configuración conjunta de la librería de compilación (opencv-4100.jar) y la librería nativa (opencv_java4100.dll). Este proceso permitió comprender la arquitectura híbrida utilizada por OpenCV, donde Java actúa como interfaz de programación mientras que las operaciones de procesamiento son ejecutadas por bibliotecas nativas optimizadas para alto rendimiento. Asimismo, se verificó la importancia de establecer correctamente el parámetro -Djava.library.path, requisito indispensable para garantizar la correcta localización y carga de las librerías durante la ejecución de las aplicaciones.

Uno de los aspectos más relevantes identificados durante el laboratorio fue la necesidad de garantizar la compatibilidad entre las versiones del entorno de desarrollo y las bibliotecas utilizadas. La aparición del error `UnsupportedClassVersionError` evidenció la importancia de verificar previamente los requisitos mínimos de software, permitiendo comprender cómo las diferencias entre versiones del JDK pueden afectar directamente la ejecución de aplicaciones modernas. La migración hacia Zulu JDK 17.0.19+10 permitió resolver exitosamente este inconveniente y asegurar la compatibilidad total con OpenCV 4.10.0, fortaleciendo conocimientos relacionados con gestión de dependencias y administración de entornos de desarrollo.

Asimismo, el laboratorio permitió identificar limitaciones prácticas relacionadas con el acceso a archivos dentro del sistema operativo Windows. Se comprobó que la utilización de rutas con caracteres especiales, tildes o símbolos propios del idioma español puede generar errores de lectura mediante funciones como `imread()`. La solución implementada consistió en reorganizar la estructura de directorios y utilizar rutas compatibles, experiencia que permitió comprender la importancia de diseñar entornos de trabajo robustos y portables para aplicaciones de visión artificial y procesamiento multimedia.

Desde el punto de vista del desarrollo de software, la solución fue implementada bajo una arquitectura modular compuesta por clases independientes para cada rutina de procesamiento. Esta estrategia facilitó la organización del código, permitió ejecutar cada práctica de forma individual mediante Apache NetBeans y favoreció la reutilización de componentes. La separación de responsabilidades aplicada durante el proyecto se encuentra alineada con principios de Ingeniería de Software relacionados con mantenibilidad, escalabilidad y claridad estructural, aspectos fundamentales en aplicaciones profesionales de procesamiento de imágenes.

En cuanto a las técnicas implementadas, el laboratorio permitió aplicar diversos algoritmos fundamentales de visión artificial. La primera rutina demostró procesos básicos de preprocesamiento mediante conversión a escala de grises, suavizado gaussiano y umbralización binaria, evidenciando cómo la transformación de imágenes puede facilitar etapas posteriores de análisis. La segunda rutina incorporó métodos de detección de bordes utilizando los algoritmos de Canny y Sobel, permitiendo identificar contornos y cambios de intensidad presentes en las imágenes, técnicas ampliamente

utilizadas en sistemas de reconocimiento de objetos, inspección industrial automatizada y análisis visual inteligente.

Adicionalmente, la implementación de la detección automática de rostros mediante clasificadores Haar Cascade permitió comprender uno de los enfoques clásicos utilizados en visión por computador para la identificación de patrones visuales. Mediante la aplicación de técnicas de ecualización del histograma y ajustes en los parámetros de sensibilidad del detector, fue posible identificar correctamente cinco rostros dentro de la imagen analizada. Este resultado evidenció la importancia de factores como la iluminación, el contraste y la calidad visual de la imagen para obtener detecciones confiables, demostrando además las capacidades de OpenCV para aplicaciones relacionadas con biometría, vigilancia inteligente y reconocimiento facial.

Por otra parte, las transformaciones geométricas implementadas en la cuarta rutina permitieron analizar operaciones fundamentales sobre imágenes digitales, incluyendo redimensionamiento, rotación y reflejos horizontales y verticales. Estas técnicas son ampliamente utilizadas en aplicaciones modernas de visión artificial, aprendizaje automático y aumento de datos para entrenamiento de modelos de inteligencia artificial, permitiendo generar variaciones controladas de una misma imagen sin alterar su contenido semántico.

Finalmente, todas las rutinas desarrolladas compilaron y ejecutaron correctamente dentro de Apache NetBeans utilizando Java 17 y OpenCV 4.10.0, generando resultados consistentes y verificables para cada uno de los procesos implementados. La correcta ejecución de las prácticas permitió validar tanto la configuración del entorno como el funcionamiento de los algoritmos aplicados, demostrando la viabilidad de utilizar OpenCV como plataforma para el desarrollo de aplicaciones de percepción computacional en Java.

En conjunto, este laboratorio constituye una base sólida para futuros desarrollos relacionados con visión artificial, reconocimiento de patrones, procesamiento digital de imágenes, aprendizaje automático e inteligencia artificial. Los conocimientos adquiridos durante la práctica fortalecen competencias fundamentales para el diseño de sistemas capaces de interpretar información visual, consolidando habilidades técnicas altamente relevantes dentro de los campos de la Ingeniería Informática, la Ciencia de Datos y la Inteligencia Artificial.

OpenCV se consolida como una de las bibliotecas más completas y ampliamente utilizadas en el ámbito de la visión artificial y el procesamiento digital de imágenes. Su integración con Java mediante Apache NetBeans proporciona un entorno de desarrollo flexible y potente para la construcción de aplicaciones inteligentes basadas en percepción visual. La amplia variedad de funciones disponibles para filtrado, detección de características, reconocimiento de patrones y análisis de imágenes convierte a OpenCV en una herramienta fundamental para el desarrollo de soluciones avanzadas en automatización, robótica, sistemas inteligentes y tecnologías basadas en inteligencia artificial.

BIBLIOGRAFÍA

A continuación, se presentan las principales fuentes bibliográficas, técnicas y académicas utilizadas como referencia para el desarrollo del presente laboratorio de Sistemas Multiagente y Percepción Computacional, orientado a la instalación, configuración y utilización de la biblioteca OpenCV en lenguaje Java para la implementación de técnicas fundamentales de procesamiento digital de imágenes y visión artificial.

Las referencias incluidas proporcionan el soporte conceptual y técnico necesario para comprender los principios relacionados con la manipulación de imágenes, filtrado digital, detección de bordes, reconocimiento de patrones, transformaciones geométricas y detección automática de rostros. Asimismo, constituyen la base teórica y práctica que permitió desarrollar las diferentes rutinas implementadas durante el laboratorio, fortaleciendo los conocimientos en áreas de visión por computador, inteligencia artificial y sistemas inteligentes basados en información visual.

- OpenCV Team. (2024). OpenCV 4.10.0 Releases. Recuperado de <https://opencv.org/releases/>
- OpenCV Team. (2024). OpenCV Java Tutorials. Recuperado de https://docs.opencv.org/4.x/d9/d52/tutorial_java_dev_intro.html

- OpenCV Team. (2024). OpenCV Documentation. Recuperado de <https://docs.opencv.org/4.x/>
- Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. Sebastopol, California: O'Reilly Media.
- Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd Edition). Springer Nature. Recuperado de <https://szeliski.org/Book/>
- Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th Edition). Pearson Education.
- Davies, E. R. (2017). Computer Vision: Principles, Algorithms, Applications, Learning (5th Edition). Academic Press.
- Sonka, M., Hlavac, V., & Boyle, R. (2014). Image Processing, Analysis, and Machine Vision (4th Edition). Cengage Learning.
- Shapiro, L. G., & Stockman, G. C. (2001). Computer Vision. Upper Saddle River, NJ: Prentice Hall.
- Jain, A. K. (1989). Fundamentals of Digital Image Processing. Englewood Cliffs, NJ: Prentice Hall.
- Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR).
- Clase magistral: Sistemas Multiagente y Percepción Computacional - Introducción a OpenCV y Visión Artificial. Fundación Universitaria Internacional de La Rioja, 2026.

AGRADECIMIENTO

Finalmente, deseo expresar mi más sincero agradecimiento al profesor Ing. Rogerio Orlando Beltrán Castro por los conocimientos, la orientación y el acompañamiento brindados durante el desarrollo de este laboratorio. Sus explicaciones y aportes en el área de percepción computacional, visión artificial y procesamiento digital de imágenes fueron fundamentales para comprender los principios teóricos y prácticos que sustentan el funcionamiento de herramientas especializadas como OpenCV.

Gracias a los conceptos compartidos durante el curso, fue posible desarrollar esta actividad de manera estructurada y técnicamente fundamentada, aplicando técnicas relacionadas con la manipulación de imágenes digitales, conversión de espacios de color, filtrado, detección de bordes, transformaciones geométricas y reconocimiento automático de rostros. Asimismo, las orientaciones proporcionadas permitieron comprender la importancia de estas tecnologías dentro del campo de la Inteligencia Artificial y su aplicación en sistemas capaces de interpretar información visual proveniente del entorno. De igual manera, esta experiencia contribuyó significativamente al fortalecimiento de mis competencias en programación orientada a objetos, desarrollo de aplicaciones en Java y utilización de bibliotecas especializadas para visión por computador. Además, permitió afianzar buenas prácticas de Ingeniería de Software relacionadas con la organización modular del código, la reutilización de componentes, la depuración de errores y la correcta configuración de entornos de desarrollo para aplicaciones de alto nivel tecnológico.

El laboratorio también permitió adquirir experiencia práctica en la integración de OpenCV con Apache NetBeans, enfrentando y resolviendo desafíos técnicos asociados a la compatibilidad de versiones, configuración de bibliotecas nativas y gestión de recursos multimedia. Estos aprendizajes fortalecen la capacidad para abordar proyectos más complejos relacionados con análisis de imágenes, reconocimiento de patrones y sistemas inteligentes basados en percepción visual.

Sin duda, el desarrollo de este laboratorio representó una experiencia académica enriquecedora que permitió consolidar conocimientos fundamentales en percepción computacional y visión artificial, estableciendo una base sólida para futuros proyectos relacionados con inteligencia artificial, aprendizaje automático, reconocimiento facial, automatización inteligente y desarrollo de sistemas capaces de interpretar y analizar información visual de manera autónoma.

¡¡¡Mil gracias profesor!!!

Respetuosamente,

Alejandro De Mendoza